

**Московский государственный технический университет
им. Н.Э. Баумана**

Утверждаю:

Большаков С.А.

"__" _____ 2025 г.

**Курсовая работа по курсу «Системное программирование»
«Резидентная программа (TSR)»**

Листинг и исходный код программ
(вид документа)

писчая бумага
(вид носителя)

38
(количество листов)

ИСПОЛНИТЕЛЬ:

студент группы ИУ5-42Б
Кудабаева Э.

"__" _____ 2025 г.

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ	2
1. ИСХОДНЫЙ КОД (KRBUFF)	2
2. ЛИСТИНГ ПРОГРАММЫ (KRBUFF)	13

1. ИСХОДНЫЙ КОД (KRBUFF)

```

; =====
; krbuff.asm
; 14.04.23
; Сборка:
; > tasm.exe /l krbuff.asm
; > tlink /t /x krbuff.obj
; =====

code segment 'code'
    assume CS:code, DS:code
    org 100h

_start:
    jmp _initKRBUFF                ; на начало программы

    ignoredChars    DB 'АБВГД'    ; русские прописные буквы - игнорируемые
                                ; символы
    ignoredLength    DB 5          ; длина строки ignoredChars
    ignoreEnabled    DB 0          ; флаг функции игнорирования ввода
    translateFrom    DB '!%';"    ; заменяемые символы
    translateTo      DB '***'     ; символы, на которые будет происходить замена
    translateLength  DB 3          ; длина строки translateFrom
    translateEnabled  DB 0          ; флаг функции перевода

    signaturePrintingEnabled DB 0    ; флаг вывода подписи
    counter          DW 0
    printDelay       EQU 3          ; задержка перед выводом "подписи" в секундах

    signatureLineLength DW 52      ; длина одной строчки подписи
    signatureLine1     DB 179, 'Кудабаева Эвита', 179
    signatureLine2     DB 179, 'ИУ5-42Б', 179
    signatureLine3     DB 179, 'Вариант #10', 179
    tableTop           DB '┌', 50 dup ('-'), '┐'
    tableBottom        DB '└', 50 dup ('-'), '┘'

    helpMsg           DB '> krbuff.com [/?]', 10, 13
                    DB ' [/?] - вывод данной справки', 10, 13
                    DB ' выгрузка резидента из памяти при повторном запуске', 10, 13
                    DB ' F7 - вывод ФИО и группы по таймеру (3 сек.) внизу экрана', 10, 13
                    DB ' F8 - включение/отключение курсивного вывода русского символа Ж', 10,
13
                    DB ' F9 - включение/отключение режима блокировки ввода букв АБВГД', 10, 13
                    DB ' F1 - включение/отключение замены символов !%; на ***', 10, 13, 0

    helpMsgLength     EQU $-helpMsg

```



```

pushf
call dword ptr CS:[old_int9hOffset] ; вызываем стандартный обработчик прерывания
mov AX, 40h ; 40h - сегмент, где хранятся флаги состояния
клавиатуры
mov ES, AX
mov BX, ES:[1Ch] ; адрес хвоста
sub BX, 2h ; сместимся назад к последнему введённому символу
cmp BX, 1Eh ; не вышли ли мы за пределы буфера?
jae _go
mov BX, 3Ch ; хвост вышел за пределы буфера: значит, последний
; введённый символ находится в конце буфера

_go:
mov DX, ES:[BX] ; в DX 0 введённый символ

_test_Fx: ; проверка F1-F9
_F1:
cmp DH, 3Bh ; F1(?)
jne _F7
not translateEnabled ; Флаг замены символов
mov ES:[1Ch], BX ; блокировка ввода символа
jmp _quit
_F7:
cmp DH, 41h ; F7(?)
jne _F8
not signaturePrintingEnabled ; Флаг печати ФИО
mov ES:[1Ch], BX ; блокировка ввода символа
jmp _quit
_F8:
cmp DH, 42h ; F8(?)
jne _F9
mov ES:[1Ch], BX ; блокировка ввода символа
not cursiveEnabled ; Установка или сброс курсива
call toggleCursive ; перевод символа в курсив и обратно
; в зависимости от флага cursiveEnabled
jmp _quit
_F9:
cmp DH, 43h ; F9(?)
jne _translateOrIgnore
not ignoreEnabled ; Включение блокировки символов
mov ES:[1Ch], BX ; блокировка ввода символа
jmp _quit

_translateOrIgnore: ; просто выводим набранный символ на экран
cmp ignoreEnabled, true ; включен ли режим блокировки ввода?
jne _checkTranslate

mov SI, 0 ; да, включен
mov CL, ignoredLength ; количество игнорируемых символов

_checkIgnored:

```

```

    cmp DL, ignoredChars[SI]          ; проверяем, присутствует ли текущий символ в
списке игнорируемых
    je _block
    inc SI
    loop _checkIgnored                ; закидываем ignoredLength раз
    jmp _checkTranslate

; блокируем
_block:
    mov ES:[1Ch], BX                  ; блокировка ввода символа
    jmp _quit

_checkTranslate:
    cmp translateEnabled, true        ; включен ли режим перевода?
    jne _quit

    mov SI, 0                         ; да, включен
    mov CL, translateLength           ; кол-во символов для перевода

_checkTranslateLoop:
    cmp DL, translateFrom[SI]         ; присутствует ли текущий символ в списке для
перевода?
    je _translate
    inc SI
    loop _checkTranslateLoop          ; продолжаем, пока не закончим проверять каждый
символ
    jmp _quit

_translate:
    xor AX, AX                        ; переводим
    mov AL, translateTo[SI]
    mov ES:[BX], AX                   ; замена символа

_quit:
    pop BP
    pop DS ES DX CX BX AX SI         ; восстанавливаем все регистры
    iret
new_int9h endp

; в зависимости от флага cursiveEnabled меняет начертание символа на курсив и обратно
; сама смена происходит в процедуре changeFont - здесь же подготавливаются данные
toggleCursive proc
    push ES AX                        ; сохраняем регистры
    push CS
    pop ES

    cmp cursiveEnabled, true          ; если флаг равен true,
    jne _restoreSymbol                ; выполняем замену символа на курсивный вариант,
; предварительно сохраняя старый символ в savedSymbol

    call saveFont
    mov CL, charToCursiveIndex

```

```

_shiftTable:
    add BP, 16                ; получаем в BP таблицу всех символов. адрес указывает
на символ 0                  ; поэтому нужно совершить сдвиг 16*X - где X - код символа

    loop _shiftTable

    push DS                  ; при savefont смещается регистр ES
    pop AX                  ; поэтому приходится делать такие махинации, чтобы
    push ES                 ; записать полученный элемент в savedSymbol
    pop DS
    push AX                 ; DS -> AX, ES -> DS, AX -> ES => ES и DS поменялись
местами
    pop ES                 ; + сохранение старого значения DS в AX
    push AX

    mov SI, BP
    lea DI, savedSymbol     ; сохраняем в переменную savedSymbol таблицу
нужного символа

    mov CX, 16              ; movsb из DS:SI в ES:DI

    rep movsb               ; исходные позиции сегментов возвращены
    pop DS                 ; восстановление DS

    mov CX, 1               ; заменим написание символа на курсив
    mov DH, 0
    mov DL, charToCursiveIndex
    lea BP, cursiveSymbol
    call changeFont
    jmp _exitToggleCursive

_restoreSymbol:
    mov CX, 1               ; если флаг равен 0, заменяем курсивный символ на
старый вариант
    mov DH, 0
    mov DL, charToCursiveIndex
    lea BP, savedSymbol
    call changeFont

_exitToggleCursive:
    pop AX
    pop ES
    ret
toggleCursive endp

; функция смены начертания символа (курсив/нормальное)
changeFont proc
    push AX BX DX
    mov AX, 1100h
    mov BX, 1000h
    int 10h
    pop DX BX AX

```

```
ret
changeFont endp
```

; функция сохранения нормального начертания символа

```
saveFont proc
    push AX BX DX
    mov AX, 1130h
    mov BX, 0600h
    int 10h
    pop BX AX DX
    ret
saveFont endp
```

; обработчик прерывания int 2Fh

; служит для:

; 1) проверки факта присутствия KRBUFF в памяти (при AH=0FFh, AL=0)

; будет возвращён AH='i' в случае, если KRBUFF уже загружен

; 2) выгрузки KRBUFF из памяти (при AH=0FFh, AL=1)

```
new_int2Fh proc
    cmp AH, 0FFh                ; наша процедура?
    jne _2Fh_default            ; нет - на стандартный обработчик
    cmp AL, 0                    ; подпроцедура проверки, загружен ли резидент в память?
    je _alreadyInstalled2Fh
    cmp AL, 1                    ; подпроцедура выгрузки из памяти?
    je _uninstall
    jmp _2Fh_default
```

_2Fh_default:

```
    jmp dword ptr CS:[old_int2FhOffset] ; вызов стандартного обработчика
```

_alreadyInstalled2Fh:

```
    mov AH, 'i'                ; пусть AH = 'i', если резидент уже загружен в память
    iret                       ; конечно, вместо 'i' может быть любое значение
```

_uninstall:

; подпроцедура выгрузки из памяти

```
    push DS ES DX BX
    xor BX, BX
```

```
    push CS                    ; CS = ES, для доступа к переменным
    pop ES
```

```
    mov AX, 2509h
    mov DX, ES:old_int9hOffset ; возвращаем вектор прерывания 09h на место
    mov DS, ES:old_int9hSegment
    int 21h
```

```
    mov AX, 251Ch
    mov DX, ES:old_int1ChOffset ; возвращаем вектор прерывания 1Ch на место
    mov DS, ES:old_int1ChSegment
    int 21h
```

```
    mov AX, 252Fh
```

```

mov DX, ES:old_int2FhOffset      ; возвращаем вектор прерывания 2Fh на место
mov DS, ES:old_int2FhSegment
int 21h

mov ES, CS:2Ch                  ; загрузим в ES адрес окружения
mov AH, 49h                     ; выгрузим из памяти окружение
int 21h
jc _notRemove

push CS
pop ES                          ; в ES - адрес резидентной программы
mov AH, 49h                     ; выгрузим из памяти резидент
int 21h

jc _notRemove
jmp _unloaded

_notRemove:                     ; не удалось выполнить выгрузку => вывод ошибки
mov AH, 03h                     ; получаем позицию курсора
int 10h
lea BP, noRemoveMsg
mov CX, noRemoveMsg_length
mov BL, 0111b
mov AX, 1301h
int 10h
jmp _2Fh_exit

_unloaded:                      ; выгрузка прошла успешно => вывод сообщения
mov AH, 03h                     ; получаем позицию курсора
int 10h
lea BP, removedMsg
mov CX, removedMsg_length
mov BL, 0111b
mov AX, 1301h
int 10h

_2Fh_exit:
pop BX DX ES DS
iret
new_int2Fh endp

; обработчик прерывания int 1Ch
; вызывается каждые 55 мс
new_int1Ch proc far
push AX
push CS
pop DS

pushf
call dword ptr CS:[old_int1ChOffset] ; вызываем стандартный обработчик
прерывания

```



```

    cmp signaturePrintingEnabled, true    ; если нажата управляющая клавиша (в данном
случае F7)
    jne _notToPrint

```

```

    cmp counter, printDelay*1000/55 + 1  ; если кол-во "тактов" равно printDelay
секундам
    je _letsPrint

```

```

    jmp _dontPrint

```

```

_letsPrint:
    not signaturePrintingEnabled
    mov counter, 0
    call printSignature                ; выводим подпись на экран

```

```

_dontPrint:
    inc counter                        ; увеличим значение счетчика на 1

```

```

_notToPrint:
    pop AX
    iret
new_int1Ch endp

```

```

; выводит одну строку подписи
printSignatureLine proc
    push DX
    mov CX, signatureLineLength
    mov BL, 0111b                ; цвет выводимого текста
    mov AX, 1301h                ; AH = 13h - номер ф-ии, AL = 01h - перемещение
курсора
    int 10h
    pop DX
    inc DH
    ret
printSignatureLine endp

```

```

; процедура вывода подписи
printSignature proc
    push AX DX CX BX ES SP BP SI DI

```

```

    xor AX, AX                    ; обнуляем значения регистров
    xor BX, BX
    xor DX, DX

```

```

    mov AH, 03h                  ; чтение текущей позиции курсора
    int 10h
    push DX                      ; помещаем информацию о положении курсора в стек

```

```

    mov DX, 130Fh                ; NB! вверху: 000Fh, посередине: 090Fh, внизу: 130Fh

```

```

_actualPrint:

```

```

    mov AH, 0Fh                ; чтение текущего видеорежима. в ВН - текущая
страница
    int 10h

    push CS
    pop ES                    ; указываем ES на CS

    lea BP, tableTop
    call printSignatureLine    ; выводим верх таблицы
    lea BP, signatureLine1
    call printSignatureLine    ; выводим первую строку
    lea BP, signatureLine2
    call printSignatureLine    ; выводим вторую строку
    lea BP, signatureLine3
    call printSignatureLine    ; выводим третью строку
    lea BP, tableBottom
    call printSignatureLine    ; выводим низ таблицы

    xor BX, BX
    pop DX                    ; восстанавливаем из стека прежнее положение курсора
    mov AH, 02h                ; меняем положение курсора на первоначальное
    int 10h

    pop DI SI BP SP ES BX CX DX AX
    ret
printSignature endp

; Основная часть программы
; 1) установка видеорежима
; 2) проверка, запущен ли резидент
; 3) установка вектора прерываний
_initKRBUFF:
    mov AH, 03h
    int 10h
    push DX
    mov AH, 00h                ; установка видеорежима
    mov AL, 83h
    int 10h
    pop DX
    mov AH, 02h
    int 10h

    call commandParamsParser    ; читаем аргументы командной строки
    cmp commandLineResult, 2    ; если результат = 2, значит была выведена
справка
    jne _shouldContinue        ; соответственно, никаких других действий делать не
нужно
    jmp _exit
_shouldContinue:
    mov AH, 0FFh
    mov AL, 0
    int 2Fh

```

```

cmp AH, 'i'                ; проверка того, загружена ли уже программа
je _remove

mov AX, 3509h              ; получить в ES:BX прерывания 09h
int 21h
mov word ptr CS:old_int9hOffset, BX ; обработчик прерывания 09h
mov word ptr CS:old_int9hSegment, ES
mov AX, 2509h              ; установим вектор на прерывание 09h
mov DX, offset new_int9h
int 21h

mov AX, 351Ch              ; получить в ES:BX прерывания 1Ch
int 21h
mov word ptr CS:old_int1ChOffset, BX ; обработчик прерывания 1Ch
mov word ptr CS:old_int1ChSegment, ES
mov AX, 251Ch              ; установим вектор на прерывание 1Ch
mov DX, offset new_int1Ch
int 21h

mov AX, 352Fh              ; получить в ES:BX прерывания 2Fh
int 21h
mov word ptr CS:old_int2FhOffset, BX ; обработчик прерывания 2Fh
mov word ptr CS:old_int2FhSegment, ES
mov AX, 252Fh              ; установим вектор на прерывание 2Fh
mov DX, offset new_int2Fh
int 21h

lea BX, installedMsg       ; выводим сообщение, что всё ОК
call printStr

mov DX, offset _initKRBUFF ; остаемся в памяти и выходим из основной
части
int 27h

_remove:                   ; выгрузка из памяти
push ES
mov AX, DS:[2Ch]           ; PSP
mov ES, AX
mov AH, 49h                ; хватит памяти чтоб остаться резидентом?
int 21h
pop ES

mov AH, 0FFh
mov AL, 1
int 2Fh
jmp _exit
_alreadyInstalled:         ; резидент уже запущен
lea BX, alreadyInstalledMsg
call printStr
jmp _exit
_exit:                    ; выход
int 20h

```

; парсер аргументов командной строки. выводит справку.
 ; устанавливает флаг commandLineResult:
 ; 0 = всё ОК; 1 = нужна выгрузка; 2 = была выведена справка, не нужно загружать
 резидент

commandParamsParser proc

push CS

pop ES

mov SI, 80h

; SI = смещение командной строки

lodsb

; получим кол-во символов

or AL, AL

; если 0 символов введено,

jz _paramParsingEnd

; то все в порядке

_nextChar:

inc SI

; теперь SI указывает на первый символ строки

cmp [SI], BYTE ptr 0

je _paramParsingEnd

lodsw

; получаем два символа

cmp AX, '?'

je _displayHelp

jmp _nextChar

_displayHelp:

lea BX, helpMsg

; выводим справку

call printStr

mov commandLineResult, 2

; флаг того, что резидент загружать не надо

_paramParsingEnd:

ret

commandParamsParser endp

; отображает символ из AL

printChar proc

mov AH, 0EH

int 010H

ret

printChar endp

; отображает нуль-терминированную строку из [BX]

printStr proc

push DX AX

mov AX, [BX]

_printStrLoop:

cmp AL, 0

je _printStrEnd

call printChar

inc BX

mov AX, [BX]

jmp _printStrLoop

```

_printStrEnd:
    pop AX DX
    ret
printStr endp

```

```

code ends
end _start

```

2. ЛИСТИНГ ПРОГРАММЫ (KRBUFF)

Turbo Assembler Version 3.1 04/14/25 23:43:56 Page 1
krbuff.asm

```

1          ; =====
2          ;   krbuff.asm
3          ; 14.04.23
4          ;   Сборка:
5          ;   > tasm.exe /l krbuff.asm
6          ;   > tlink /t /x krbuff.obj
7          ; =====
8
9 0000      code segment 'code'
10          assume      CS:code, DS:code
11          org 100h
12
13 0100      _start:
14 0100 E9 0568      jmp _initKRBUFF          ; на начало
программы
15
16 0103 80 81 82 83 84      ignoredChars      DB 'АБВГД'          ; русские
прописные буквы - игнорируемые символы
17 0108 05                  ignoredLength      DB 5              ; длина
строки ignoredChars
18 0109 00                  ignoreEnabled      DB 0              ; флаг
функции игнорирования ввода
19 010A 21 25 3B                  translateFrom      DB "!"%;"      ;
заменяемые символы
20 010D 2A 2A 2A                  translateTo      DB '***'          ; символы,
на которые будет происходить замена
21 0110 03                  translateLength      DB 3              ; длина строки
translateFrom
22 0111 00                  translateEnabled      DB 0              ; флаг функции
перевода
23
24 0112 00                  signaturePrintingEnabled DB 0          ; флаг
вывода подписи
25 0113 0000                  counter            DW 0
26      =0003                  printDelay        EQU 3              ; задержка перед
выводом "подписи" в секундах
27

```

28	0115 0034	signatureLineLength	DW 52	; длина одной
строчки подписи				
29	0117 B3 8A E3 A4 A0 A1	A0+ signatureLine1		DB 179,
'Кудабаева Эвита		' , +		
30	A5 A2 A0 20 9D A2	A8+ 179		
31	E2 A0 20 20 20 20	20+		
32	20 20 20 20 20 20 20+			
33	20 20 20 20 20 20 20+			
34	20 20 20 20 20 20 20+			
35	20 20 20 20 20 20 20+			
36	20 20 B3			
37	014B B3 88 93 35 2D 34	32+ signatureLine2		DB 179, 'ИУ5-42Б
		' , +		
38	81 20 20 20 20 20 20+	179		
39	20 20 20 20 20 20 20+			
40	20 20 20 20 20 20 20+			
41	20 20 20 20 20 20 20+			
42	20 20 20 20 20 20 20+			
43	20 20 20 20 20 20 20+			
44	20 20 B3			
45	017F B3 82 A0 E0 A8 A0	AD+ signatureLine3		DB 179, 'Вариант
#10		' , +		
46	E2 20 23 31 30 20	20+ 179		
47	20 20 20 20 20 20 20+			
48	20 20 20 20 20 20 20+			
49	20 20 20 20 20 20 20+			
50	20 20 20 20 20 20 20+			
51	20 20 20 20 20 20 20+			
52	20 20 B3			
53	01B3 DA 32*(C4) BF	tableTop		DB '┐', 50 dup ('─'), '┐'
54	01E7 C0 32*(C4) D9	tableBottom		DB '└', 50 dup ('─'), '└'
55				
56	021B 3E 20 6B 72 62 75	66+ helpMsg	DB	'> krbuff.com [/?] ' , 10, 13
57	66 2E 63 6F 6D 20	5B+		

Turbo Assembler Version 3.1 04/14/25 23:43:56 Page 2
krbuff.asm

```

58      2F 3F 5D 20 0A 0D
59 022F 20 5B 2F 3F 5D 20 2D+      DB ' [/?] - вывод данной справки',
      10, 13
60      20 A2 EB A2 AE A4 20+
61      A4 A0 AD AD AE A9 20+
62      E1 AF E0 A0 A2 AA A8+
63      0A 0D
64 024D 20 A2 EB A3 E0 E3 A7+      DB ' выгрузка резидента из      памяти
при повторном запуске', 10, 13
65      AA A0 20 E0 A5 A7 A8+
66      A4 A5 AD E2 A0 20 A8+
67      A7 20 AF A0 AC EF E2+
68      A8 20 AF E0 A8 20 AF+
69      AE A2 E2 AE E0 AD AE+
70      AC 20 A7 A0 AF E3 E1+
71      AA A5 0A 0D
72 0282 20 20 46 37 20 20 2D+      DB ' F7 - вывод ФИО и группы по
таймеру (3 сек.)      внизу экрана', 10, 13
73      20 A2 EB A2 AE A4 20+
74      94 88 8E 20 A8 20 A3+
75      E0 E3 AF AF EB 20 AF+
76      AE 20 E2 A0 A9 AC A5+
77      E0 E3 20 28 33 20 E1+
78      A5 AA 2E 29 20 A2 AD+
79      A8 A7 E3 20 ED AA E0+
80      A0 AD A0 0A 0D
81 02BF 20 20 46 38 20 20 2D+      DB ' F8 - включение/отключение
курсивного вывода русского символа Ж', 10, 13
82      20 A2 AA AB EE E7 A5+
83      AD A8 A5 2F AE E2 AA+
84      AB EE E7 A5 AD A8 EF+
85      20 AA E3 E0 E1 A8 A2+
86      AD AE A3 AE 20 A2 EB+
87      A2 AE A4 A0 20 E0 E3+
88      E1 E1 AA AE A3 AE 20+
89      E1 A8 AC A2 AE AB A0+
90      20 86 0A 0D
91 0302 20 20 46 39 20 20 2D+      DB ' F9 - включение/отключение
режима блокировки ввода букв АБВГД', 10, 13
92      20 A2 AA AB EE E7 A5+
93      AD A8 A5 2F AE E2 AA+
94      AB EE E7 A5 AD A8 A5+
95      20 E0 A5 A6 A8 AC A0+
96      20 A1 AB AE AA A8 E0+
97      AE A2 AA A8 20 A2 A2+
98      AE A4 A0 20 A1 E3 AA+
99      A2 20 80 81 82 83 84+

```

100	0A 0D		
101	0343 20 20 46 31 20 20	2D+	DB ' F1 - включение/отключение
замены символов !%; на ***, 10, 13, 0			
102	20 A2 AA AB EE E7	A5+	
103	AD A8 A5 2F AE E2	AA+	
104	AB EE E7 A5 AD A8	A5+	
105	20 A7 A0 AC A5 AD	EB+	
106	20 E1 A8 AC A2 AE	AB+	
107	AE A2 20 21 25 3B	20+	
108	AD A0 20 2A 2A 2A	0A+	
109	0D 00		
110			
111	=0162	helpMsgLength	EQU \$-helpMsg
112	037D 00	commandLineResult	DB 0
113			
114	037E 00	cursiveEnabled	DB 0 ; флаг

перевода символа в курсив

Turbo Assembler Version 3.1 04/14/25 23:43:56 Page 3
krbuff.asm

```

115 037F 00                                cursiveSymbol                DB 00000000b    ; символ,
составленный из единиц (его курсивный+
116                                вариант)
117 0380 00                                DB 00000000b
118 0381 00                                DB 00000000b
119 0382 63                                DB 01100011b
120 0383 63                                DB 01100011b
121 0384 77                                DB 01110111b
122 0385 5A                                DB 01011010b
123 0386 66                                DB 01100110b
124 0387 66                                DB 01100110b
125 0388 66                                DB 01100110b
126 0389 CC                                DB 11001100b
127 038A CC                                DB 11001100b
128 038B CC                                DB 11001100b
129 038C 00                                DB 00000000b
130 038D 00                                DB 00000000b
131 038E 00                                DB 00000000b
132
133 038F 86                                charToCursiveIndex          DB 'Ж'          ; символ для
замены
134 0390 10*(FF)                          savedSymbol                  DB 16 dup(0FFh) ; переменная
для хранения старого символа
135
136 03A0 ????                                old_int9hOffset              DW ?              ; адрес старого
обработчика int 9h
137 03A2 ????                                old_int9hSegment             DW ?              ; сегмент старого
обработчика int 9h
138 03A4 ????                                old_int1ChOffset             DW ?              ; адрес старого
обработчика int 1Ch
139 03A6 ????                                old_int1ChSegment            DW ?              ; сегмент старого
обработчика int 1Ch
140 03A8 ????                                old_int2FhOffset             DW ?              ; адрес старого
обработчика int 2Fh
141 03AA ????                                old_int2FhSegment            DW ?              ; сегмент старого
обработчика int 2Fh
142
143 03AC 90 A5 A7 A8 A4 A5 AD+  installedMsg                DB 'Резидент загружен.',
0
144    E2 20 A7 A0 A3 E0    E3+
145    A6 A5 AD 2E 00
146 03BF 90 A5 A7 A8 A4 A5 AD+  alreadyInstalledMsg          DB 'Резидент уже был
загружен.', 0
147    E2 20 E3 A6 A5 20    A1+
148    EB AB 20 A7 A0 A3    E0+
149    E3 A6 A5 AD 2E 00

```

```

150 03DA 90 A5 A7 A8 A4 A5 AD+ notInstalledMsg DB 'Резидент не был
загружен.$'
151 E2 20 AD A5 20 A1 EB+
152 AB 20 A7 A0 A3 E0 E3+
153 A6 A5 AD 2E 24
154
155 03F4 90 A5 A7 A8 A4 A5 AD+ removedMsg DB 'Резидент выгружен
из памяти.'
156 E2 20 A2 EB A3 E0 E3+
157 A6 A5 AD 20 A8 A7 20+
158 AF A0 AC EF E2 A8 2E
159 =001C removedMsg_length EQU $-removedMsg
160
161 0410 8D A5 20 E3 A4 A0 AB+ noRemoveMsg DB 'Не удалось
выгрузить резидент'
162 AE E1 EC 20 A2 EB A3+
163 E0 E3 A7 A8 E2 EC 20+
164 E0 A5 A7 A8 A4 A5 AD+
165 E2
166 =001D noRemoveMsg_length EQU $-noRemoveMsg
167
168 =00FF true EQU 0FFh ; нужно для
удобства использования not с флагами ; 0FFh = 11111111b
169
= инверсия 00000000b
170
171 ; новый обработчик прерывания int 9h

```

Turbo Assembler Version 3.1 04/14/25 23:43:56 Page 4
krbuff.asm

```

172                                ; (работа с клавиатурой)
173 042D                                new_int9h proc      far
174 042D 56 50 53 51 52 06    1E      push SI AX      BX CX DX ES DS
; сохраняем значения всех, изменяемых регистров+
175                                в стеке
176 0434 55                                push BP
177 0435 0E                                push CS                                ; синхронизируем
CS и DS
178 0436 1F                                pop DS
179
180 0437 9C                                pushf
181 0438 2E: FF 1E    03A0r      call dword ptr CS:[old_int9hOffset] ;
вызываем стандартный обработчик прерывания
182 043D B8 0040                                mov AX, 40h                                ; 40h -
сегмент, где хранятся флаги состояния +
183                                клавиатуры
184 0440 8E C0                                mov ES, AX
185 0442 26: 8B 1E    001C      mov BX, ES:[1Ch]                                ;
адрес хвоста
186 0447 83 EB 02                                sub BX, 2h                                ; сместимся
назад к последнему введённому +
187                                символу
188 044A 83 FB 1E                                cmp BX, 1Eh                                ; не вышли
ли мы за пределы буфера?
189 044D 73 03                                jae _go
190 044F BB 003C                                mov BX, 3Ch                                ; хвост
вышел за пределы буфера: значит, +
191                                последний
192                                ; введённый
символ находится в конце буфера
193 0452                                _go:
194 0452 26: 8B 17                                mov DX, ES:[BX]                                ; в DX 0
введённый символ
195
196 0455                                _test_Fx:                                ; проверка F1-F9
197 0455                                _F1:
198 0455 80 FE 3B                                cmp DH, 3Bh                                ; F1(?)
199 0458 75 0C                                jne _F7
200 045A F6 16 0111r      not translateEnabled                                ; Флаг замены
символов
201 045E 26: 89 1E    001C      mov ES:[1Ch], BX                                ;
блокировка ввода символа
202 0463 EB 7C 90                                jmp _quit
203 0466                                _F7:
204 0466 80 FE 41                                cmp DH, 41h                                ; F7(?)
205 0469 75 0C                                jne _F8

```

206 046B F6 16 0112r	not signaturePrintingEnabled	; Флаг печати
ФИО		
207 046F 26: 89 1E 001C	mov ES:[1Ch], BX	;
блокировка ввода символа		
208 0474 EB 6B 90	jmp _quit	
209 0477	_F8:	
210 0477 80 FE 42	cmp DH, 42h	; F8(?)
211 047A 75 0F	jne _F9	
212 047C 26: 89 1E 001C	mov ES:[1Ch], BX	;
блокировка ввода символа		
213 0481 F6 16 037Er	not cursiveEnabled	; Установка
или сброс курсива		
214 0485 E8 0062	call toggleCursive	; перевод символа
в курсив и обратно		
215		; в зависимости от
флага cursiveEnabled		
216 0488 EB 57 90	jmp _quit	
217 048B	_F9:	
218 048B 80 FE 43	cmp DH, 43h	; F9(?)
219 048E 75 0C	jne _translateOrIgnore	
220 0490 F6 16 0109r	not ignoreEnabled	; Включение
блокировки символов		
221 0494 26: 89 1E 001C	mov ES:[1Ch], BX	;
блокировка ввода символа		
222 0499 EB 46 90	jmp _quit	
223		
224 049C	_translateOrIgnore:	; просто выводим
набранный символ на экран		
225 049C 80 3E 0109r FF	cmp ignoreEnabled, true	;
включен ли режим блокировки ввода?		
226 04A1 75 1B	jne _checkTranslate	
227		
228 04A3 BE 0000	mov SI, 0	; да,
включен		

Turbo Assembler Version 3.1 04/14/25 23:43:56 Page 5
krbuff.asm

```

229 04A6 8A 0E 0108r      mov CL, ignoredLength      ; количество
игнорируемых символов
230
231 04AA                  _checkIgnored:
232 04AA 3A 94 0103r      cmp DL, ignoredChars[SI]      ; проверяем,
присутствует ли текущий символ в +
233                        списке игнорируемых
234 04AE 74 06            je _block
235 04B0 46                inc SI
236 04B1 E2 F7            loop _checkIgnored      ;
зацикливаем ignoredLength раз
237 04B3 EB 09 90          jmp _checkTranslate
238
239                        ; блокируем
240 04B6                  _block:
241 04B6 26: 89 1E 001C      mov ES:[1Ch], BX      ;
блокировка ввода символа
242 04BB EB 24 90          jmp _quit
243
244 04BE                  _checkTranslate:
245 04BE 80 3E 0111r FF      cmp translateEnabled, true      ;
включен ли режим перевода?
246 04C3 75 1C            jne _quit
247
248 04C5 BE 0000            mov SI, 0      ; да,
включен
249 04C8 8A 0E 0110r      mov CL, translateLength      ; кол-во символов
для перевода
250
251 04CC                  _checkTranslateLoop:
252 04CC 3A 94 010Ar      cmp DL, translateFrom[SI]      ;
присутствует ли текущий символ в списке для +
253                        перевода?
254 04D0 74 06            je _translate
255 04D2 46                inc SI
256 04D3 E2 F7            loop _checkTranslateLoop      ;
продолжаем, пока не закончим проверять каждый+
257                        символ
258 04D5 EB 0A 90          jmp _quit
259
260 04D8                  _translate:
261 04D8 33 C0            xor AX, AX      ; переводим
262 04DA 8A 84 010Dr      mov AL, translateTo[SI]
263 04DE 26: 89 07          mov ES:[BX], AX      ; замена
символа
264
265 04E1                  _quit:

```

```

266 04E1 5D                pop BP
267 04E2 1F 07 5A 59 5B 58 5E    pop DS ES DX CX BX  AX SI      ;
восстанавливаем все регистры
268 04E9 CF                iret
269 04EA                    new_int9h endp
270
271                        ; в зависимости от флага cursiveEnabled меняет
начертание символа на курсив и обратно
272                        ; сама смена происходит в процедуре changeFont -
здесь же подготавливаются данные
273 04EA                    toggleCursive proc
274 04EA 06 50              push ES AX                ; сохраняем
регистры
275 04EC 0E                push CS
276 04ED 07                pop ES
277
278 04EE 80 3E 037Er FF      cmp cursiveEnabled, true        ; если флаг
равен true,
279 04F3 75 30              jne _restoreSymbol        ; выполняем
замену символа на курсивный вариант,
280                        ; предварительно
сохраняя старый символ в +
281                        savedSymbol
282
283 04F5 E8 004E            call saveFont
284 04F8 8A 0E 038Fr        mov CL, charToCursiveIndex
285 04FC                    _shiftTable:

```

```

286 04FC 83 C5 10          add BP, 16          ; получаем в
BP таблицу всех символов. адрес +
287                      указывает на символ 0
288                      ; поэтому нужно
совершить сдвиг 16*X - где X - +
289                      код символа
290 04FF E2 FB          loop _shiftTable
291
292 0501 1E          push DS          ; при savefont
смещается регистр ES
293 0502 58          pop AX          ; поэтому
приходится делать такие махинации, +
294                      чтобы
295 0503 06          push ES          ; записать
полученный элемент в savedSymbol
296 0504 1F          pop DS
297 0505 50          push AX          ; DS -> AX, ES ->
DS, AX -> ES => ES и DS +
298                      поменялись местами
299 0506 07          pop ES          ; + сохранение
старого значения DS в AX
300 0507 50          push AX
301
302 0508 8B F5          mov SI, BP
303 050A BF 0390r      lea DI, savedSymbol      ; сохраняем
в переменную savedSymbol таблицу +
304                      нужного символа
305
306 050D B9 0010      mov CX, 16          ;
movsb из DS:SI в ES:DI
307
308 0510 F3> A4      rep movsb          ; исходные
позиции сегментов возвращены
309 0512 1F          pop DS          ; восстановление
DS
310
311 0513 B9 0001      mov CX, 1          ; заменим
написание символа на курсив
312 0516 B6 00          mov DH, 0
313 0518 8A 16 038Fr  mov DL, charToCursiveIndex
314 051C BD 037Fr      lea BP, cursiveSymbol
315 051F E8 0015      call changeFont
316 0522 EB 10 90      jmp _exitToggleCursive
317
318 0525          _restoreSymbol:
319 0525 B9 0001      mov CX, 1          ; если флаг
равен 0, заменяем курсивный символ +

```

320		на старый вариант
321 0528 B6 00	mov DH, 0	
322 052A 8A 16 038Fr	mov DL, charToCursiveIndex	
323 052E BD 0390r	lea BP, savedSymbol	
324 0531 E8 0003	call changeFont	
325		
326 0534	_exitToggleCursive:	
327 0534 58	pop AX	
328 0535 07	pop ES	
329 0536 C3	ret	
330 0537	toggleCursive endp	
331		
332		; функция смены начертания символа
(курсив/нормальное)		
333 0537	changeFont proc	
334 0537 50 53 52	push AX BX DX	
335 053A B8 1100	mov AX, 1100h	
336 053D BB 1000	mov BX, 1000h	
337 0540 CD 10	int 10h	
338 0542 5A 5B 58	pop DX BX AX	
339 0545 C3	ret	
340 0546	changeFont endp	
341		
342		; функция сохранения нормального начертания
символа		

Turbo Assembler Version 3.1 04/14/25 23:43:56 Page 7
krbuff.asm

```

343 0546                                saveFont proc
344 0546 50 53 52                        push AX BX      DX
345 0549 B8 1130                        mov AX, 1130h
346 054C BB 0600                        mov BX, 0600h
347 054F CD 10                          int 10h
348 0551 5B 58 5A                      pop BX AX DX
349 0554 C3                            ret
350 0555                                saveFont endp
351
352                                ; обработчик прерывания int 2Fh
353                                ; служит для:
354                                ; 1) проверки факта присутствия KRBUFF в
памяти (при AH=0FFh, AL=0)
355                                ; будет возвращён AH='i' в случае, если
KRBUFF уже загружен
356                                ; 2) выгрузки KRBUFF из памяти (при AH=0FFh,
AL=1)
357 0555                                new_int2Fh proc
358 0555 80 FC FF                        cmp AH, 0FFh ; наша
процедура?
359 0558 75 0B                          jne _2Fh_default ; нет - на
стандартный обработчик
360 055A 3C 00                          cmp AL, 0 ; подпроцедура
проверки, загружен ли резидент в+
361                                память?
362 055C 74 0C                          je _alreadyInstalled2Fh
363 055E 3C 01                          cmp AL, 1 ; подпроцедура
выгрузки из памяти?
364 0560 74 0B                          je _uninstall
365 0562 EB 01 90                      jmp _2Fh_default
366
367 0565                                _2Fh_default:
368 0565 2E FF 2E 03A8r                jmp dword ptr CS:[old_int2FhOffset] ; вызов
стандартного обработчика
369
370 056A                                _alreadyInstalled2Fh:
371 056A B4 69                          mov AH, 'i' ; пусть AH= 'i', если
резидент уже загружен в +
372                                память
373 056C CF                            iret ; конечно, вместо 'i'
может быть любое значение
374
375 056D                                _uninstall: ; подпроцедура
выгрузки из памяти
376 056D 1E 06 52 53                  push DS ES      DX BX
377 0571 33 DB                        xor BX, BX
378

```

379 0573 0E	push CS	; CS = ES, для
доступа к переменным		
380 0574 07	pop ES	
381		
382 0575 B8 2509	mov AX, 2509h	
383 0578 26: 8B 16 03A0r	mov DX, ES:old_int9hOffset	;
возвращаем вектор прерывания 09h	на место	
384 057D 26: 8E 1E 03A2r	mov DS, ES:old_int9hSegment	
385 0582 CD 21	int 21h	
386		
387 0584 B8 251C	mov AX, 251Ch	
388 0587 26: 8B 16 03A4r	mov DX, ES:old_int1ChOffset	;
возвращаем вектор прерывания 1Ch	на место	
389 058C 26: 8E 1E 03A6r	mov DS, ES:old_int1ChSegment	
390 0591 CD 21	int 21h	
391		
392 0593 B8 252F	mov AX, 252Fh	
393 0596 26: 8B 16 03A8r	mov DX, ES:old_int2FhOffset	;
возвращаем вектор прерывания 2Fh	на место	
394 059B 26: 8E 1E 03AAr	mov DS, ES:old_int2FhSegment	
395 05A0 CD 21	int 21h	
396		
397 05A2 2E: 8E 06 002C	mov ES, CS:2Ch	; загрузим в
ES адрес окружения		
398 05A7 B4 49	mov AH, 49h	; выгрузим из
памяти окружение		
399 05A9 CD 21	int 21h	

Turbo Assembler Version 3.1 04/14/25 23:43:56 Page 8
krbuff.asm

```

400 05AB 72 0B          jc  _notRemove
401
402 05AD 0E          push CS
403 05AE 07          pop ES                      ; в ES - адрес
резидентной программы
404 05AF B4 49          mov AH, 49h                      ; выгрузимиз
памяти резидент
405 05B1 CD 21          int 21h
406
407 05B3 72 03          jc  _notRemove
408 05B5 EB 15 90          jmp _unloaded
409
410 05B8              _notRemove:                      ; не удалось
выполнить выгрузку => вывод ошибки
411 05B8 B4 03          mov AH, 03h                      ; получаем
    позицию курсора
412 05BA CD 10          int 10h
413 05BC BD 0410r          lea BP, noRemoveMsg
414 05BF B9 001D          mov CX, noRemoveMsg_length
415 05C2 B3 07          mov BL, 0111b
416 05C4 B8 1301          mov AX, 1301h
417 05C7 CD 10          int 10h
418 05C9 EB 12 90          jmp _2Fh_exit
419
420 05CC              _unloaded:                      ; выгрузка прошла
успешно => вывод сообщения
421 05CC B4 03          mov AH, 03h                      ; получаем
    позицию курсора
422 05CE CD 10          int 10h
423 05D0 BD 03F4r          lea BP, removedMsg
424 05D3 B9 001C          mov CX, removedMsg_length
425 05D6 B3 07          mov BL, 0111b
426 05D8 B8 1301          mov AX, 1301h
427 05DB CD 10          int 10h
428
429 05DD              _2Fh_exit:
430 05DD 5B 5A 07 1F          pop BX DX ES DS
431 05E1 CF          iret
432 05E2          new_int2Fh endp
433
434                      ; обработчик прерывания int 1Ch
435                      ; вызывается каждые 55 мс
436 05E2          new_int1Ch proc far
437 05E2 50          push AX
438 05E3 0E          push CS
439 05E4 1F          pop DS
440

```

```

441 05E5 9C                                pushf
442 05E6 2E: FF 1E    03A4r                call dword ptr CS:[old_int1ChOffset] ;
вызываем стандартный обработчик прерывания
443
444 05EB 80 3E 0112r FF                    cmp signaturePrintingEnabled, true ;
если нажата управляющая клавиша (в данном +
445                                     случае F7)
446 05F0 75 1B                                jne _notToPrint
447
448 05F2 83 3E 0113r 37                    cmp counter, printDelay*1000/55 + 1 ; если кол-во
"тактов" равно printDelay секундам
449 05F7 74 03                                je _letsPrint
450
451 05F9 EB 0E 90                                jmp _dontPrint
452
453 05FC                                     _letsPrint:
454 05FC F6 16 0112r                not signaturePrintingEnabled
455 0600 C7 06 0113r 0000                mov counter, 0
456 0606 E8 0016                    call printSignature ; выводим подпись на
экран

```

Turbo Assembler Version 3.1 04/14/25 23:43:56 Page 9
krbuff.asm

```

457
458 0609                _dontPrint:
459 0609 FF 06 0113r      inc counter                ; увеличим
                     значение счетчика на 1
460
461 060D                _notToPrint:
462 060D 58              pop AX
463 060E CF             iret
464 060F                new_int1Ch endp
465
466                    ; выводит одну строку подписи
467 060F                printSignatureLine proc
468 060F 52              push DX
469 0610 8B 0E 0115r      mov CX, signatureLineLength
470 0614 B3 07           mov BL, 0111b                ; цвет выводимого
текста
471 0616 B8 1301         mov AX, 1301h                ; AH = 13h-
номер ф-ии, AL = 01h - перемещение+
472                    курсора
473 0619 CD 10           int 10h
474 061B 5A             pop DX
475 061C FE C6          inc DH
476 061E C3            ret
477 061F                printSignatureLine endp
478
479                    ; процедура вывода подписи
480 061F                printSignature proc
481 061F 50 52 51 53 06 54 55+ push AX DX CX BX ES SP BP SI DI
482 56 57
483
484 0628 33 C0           xor AX, AX                    ; обнуляем
                     значения регистров
485 062A 33 DB           xor BX, BX
486 062C 33 D2           xor DX, DX
487
488 062E B4 03           mov AH, 03h                ; чтение текущей
позиции курсора
489 0630 CD 10           int 10h
490 0632 52             push DX                    ; помещаем
                     информацию о положении курсора в стек
491
492 0633 BA 130F         mov DX, 130Fh                ; NB!
вверху: 000Fh, посередине: 090Fh, внизу: +
493                    130Fh
494
495 0636                _actualPrint:

```

496 0636 B4 0F	mov AH, 0Fh	; чтение текущего
видеорежима. в BH - текущая +		
497	страница	
498 0638 CD 10	int 10h	
499		
500 063A 0E	push CS	
501 063B 07	pop ES	; указываем ES на
CS		
502		
503 063C BD 01B3r	lea BP, tableTop	
504 063F E8 FFCD	call printSignatureLine	; выводим
верх таблицы		
505 0642 BD 0117r	lea BP, signatureLine1	
506 0645 E8 FFC7	call printSignatureLine	; выводим
первую строку		
507 0648 BD 014Br	lea BP, signatureLine2	
508 064B E8 FFC1	call printSignatureLine	; выводим
вторую строку		
509 064E BD 017Fr	lea BP, signatureLine3	
510 0651 E8 FFBB	call printSignatureLine	; выводим
третью строку		
511 0654 BD 01E7r	lea BP, tableBottom	
512 0657 E8 FFB5	call printSignatureLine	; выводим
низ таблицы		
513		

Turbo Assembler Version 3.1 04/14/25 23:43:56 Page 10
krbuff.asm

```

514 065A 33 DB          xor BX, BX
515 065C 5A             pop DX          ; восстанавливаем
из стека    прежнее    положение +
516             курсора
517 065D B4 02          mov AH, 02h      ; меняем
положение  курсора    на первоначальное
518 065F CD 10          int 10h
519
520 0661 5F 5E 5D 5C 07 5B 59+ pop DI SI BP SP ES    BX CX DX AX
521    5A 58
522 066A C3             ret
523 066B             printSignature      endp
524
525             ; Основная часть программы
526             ; 1) установка видеорежима
527             ; 2) проверка, запущен ли резидент
528             ; 3) установка вектора прерываний
529 066B             _initKRBUF:
530 066B B4 03          mov AH, 03h
531 066D CD 10          int 10h
532 066F 52             push DX
533 0670 B4 00          mov AH, 00h      ; установка
видеорежима
534 0672 B0 83          mov AL, 83h
535 0674 CD 10          int 10h
536 0676 5A             pop DX
537 0677 B4 02          mov AH, 02h
538 0679 CD 10          int 10h
539
540 067B E8 0084         call commandParamsParser      ;
читаем аргументы  командной строки
541 067E 80 3E 037Dr 02 cmp commandLineResult, 2      ;
если результат = 2, значит была выведена +
542             справка
543 0683 75 03          jne _shouldContinue      ; соответственно,
никаких других действий +
544             делать не нужно
545 0685 EB 79 90        jmp _exit
546 0688             _shouldContinue:
547 0688 B4 FF          mov AH, 0FFh
548 068A B0 00          mov AL, 0
549 068C CD 2F          int 2Fh
550 068E 80 FC 69        cmp AH, 'i'            ; проверка
того, загружена ли уже программа
551 0691 74 50          je _remove
552

```

553 0693 B8 3509	mov AX, 3509h	; получить в
ES:BX прерывания 09h		
554 0696 CD 21	int 21h	
555 0698 2E: 89 1E 03A0r	mov word ptr CS:old_int9hOffset, BX	;
обработчик прерывания 09h		
556 069D 2E: 8C 06 03A2r	mov word ptr CS:old_int9hSegment, ES	
557 06A2 B8 2509	mov AX, 2509h	; установим
вектор на прерывание 09h		
558 06A5 BA 042Dr	mov DX, offset new_int9h	
559 06A8 CD 21	int 21h	
560		
561 06AA B8 351C	mov AX, 351Ch	; получить в
ES:BX прерывания 1Ch		
562 06AD CD 21	int 21h	
563 06AF 2E: 89 1E 03A4r	mov word ptr CS:old_int1ChOffset, BX	;
обработчик прерывания 1Ch		
564 06B4 2E: 8C 06 03A6r	mov word ptr CS:old_int1ChSegment, ES	
565 06B9 B8 251C	mov AX, 251Ch	; установим
вектор на прерывание 1Ch		
566 06BC BA 05E2r	mov DX, offset new_int1Ch	
567 06BF CD 21	int 21h	
568		
569 06C1 B8 352F	mov AX, 352Fh	; получить в
ES:BX прерывания 2Fh		
570 06C4 CD 21	int 21h	

Turbo Assembler Version 3.1 04/14/25 23:43:56 Page 11
krbuff.asm

```

571 06C6 2E: 89 1E      03A8r      mov word ptr CS:old_int2FhOffset, BX  ;
обработчик прерывания 2Fh
572 06CB 2E: 8C 06      03AAr      mov word ptr CS:old_int2FhSegment, ES
573 06D0 B8 252F              mov AX, 252Fh                      ; установим
векторна прерывание 2Fh
574 06D3 BA 0555r              mov DX, offset new_int2Fh
575 06D6 CD 21              int 21h
576
577 06D8 BB 03ACr              lea BX, installedMsg                ; выводим
сообщение, что всё ОК
578 06DB E8 004D              call printStr
579
580 06DE BA 066Br              mov DX, offset _initKRBUFF          ;
остаемся в памяти и выходим из основной +
581                                части
582 06E1 CD 27              int 27h
583
584 06E3              _remove:                          ; выгрузка из
памяти
585 06E3 06              push ES
586 06E4 A1 002C              mov AX, DS:[2Ch]                    ; PSP
587 06E7 8E C0              mov ES, AX
588 06E9 B4 49              mov AH, 49h                        ; хватит памяти
чтоб остаться резидентом?
589 06EB CD 21              int 21h
590 06ED 07              pop ES
591
592 06EE B4 FF              mov AH, 0FFh
593 06F0 B0 01              mov AL, 1
594 06F2 CD 2F              int 2Fh
595 06F4 EB 0A 90              jmp _exit
596 06F7              _alreadyInstalled:                ; резидент уже
запущен
597 06F7 BB 03BFr              lea BX, alreadyInstalledMsg
598 06FA E8 002E              call printStr
599 06FD EB 01 90              jmp _exit
600 0700              _exit:                          ; выход
601 0700 CD 20              int 20h
602
603                                ; парсер аргментов командной строки. выводит
справку.
604                                ; устанавливает флаг commandLineResult:
605                                ; 0 = всё ОК; 1 = нужна выгрузка; 2 = была выведена
справка, не нужно загружать резидент
606 0702              commandParamsParser proc
607 0702 0E              push CS
608 0703 07              pop ES

```

```

609
610 0704 BE 0080          mov SI, 80h          ; SI =
смещение командной строки
611 0707 AC              lodsb              ; получим кол-во
символов
612 0708 0A C0          or  AL, AL          ; если 0
символов введено,
613 070A 74 19          jz  _paramParsingEnd  ; то все в порядке
614
615 070C                _nextChar:
616 070C 46              inc SI              ; теперь SI
указывает на первый символ строки
617
618 070D 80 3C 00        cmp [SI], BYTE ptr 0
619 0710 74 13          je  _paramParsingEnd
620
621 0712 AD              lodsw              ; получаем два
символа
622 0713 3D 3F2F        cmp AX, '?'
623 0716 74 02          je  _displayHelp
624 0718 EB F2          jmp _nextChar
625
626 071A                _displayHelp:
627 071A BB 021Br        lea BX, helpMsg      ; выводим
справку

```

Turbo Assembler Version 3.1 04/14/25 23:43:56 Page 12
 krbuff.asm

```

628 071D E8 000B          call printStr
629 0720 C6 06 037Dr 02   mov commandLineResult, 2      ;
флаг того, что резидент загружать не надо
630
631 0725                  _paramParsingEnd:
632 0725 C3              ret
633 0726                  commandParamsParser endp
634
635                      ; отображает символ из    AL
636 0726                  printChar proc
637 0726 B4 0E          mov AH, 0EH
638 0728 CD 10          int 010H
639 072A C3              ret
640 072B                  printChar endp
641
642                      ; отображает нуль-терминированную строку из [BX]
643 072B                  printStr proc
644 072B 52 50          push DX AX
645 072D 8B 07          mov AX, [BX]
646 072F              _printStrLoop:
647 072F 3C 00          cmp AL, 0
648 0731 74 08          je  _printStrEnd
649 0733 E8 FFF0        call printChar
650 0736 43              inc BX
651 0737 8B 07          mov AX, [BX]
652 0739 EB F4          jmp _printStrLoop
653 073B              _printStrEnd:
654 073B 58 5A          pop AX DX
655 073D C3              ret
656 073E                  printStr endp
657
658 073E                  code ends
659                      end _start

```

Symbol Name	Type	Value
??DATE	Text	"04/14/25"
??FILENAME	Text	"krbuff "
??TIME	Text	"23:43:56"
??VERSION	Number	030A
@CPU	Text	0101H
@CURSEG	Text	CODE
@FILENAME	Text	KRBUFF
@WORDSIZE	Text	2
ALREADYINSTALLEDMSG	Byte	CODE:03BF
CHANGEFONT	Near	CODE:0537
CHARTOCURSIVEINDEX	Byte	CODE:038F
COMMANDLINERESULT	Byte	CODE:037D
COMMANDPARAMSPARSER	Near	CODE:0702
COUNTER	Word	CODE:0113
CURSIVEENABLED	Byte	CODE:037E
CURSIVESYMBOL	Byte	CODE:037F
HELPMMSG	Byte	CODE:021B
HELPMMSGLENGTH	Number	0162
IGNOREDCHARS	Byte	CODE:0103
IGNOREDLENGTH	Byte	CODE:0108
IGNOREENABLED	Byte	CODE:0109
INSTALLEDMSG	Byte	CODE:03AC
NEW_INT1CH	Far	CODE:05E2
NEW_INT2FH	Near	CODE:0555
NEW_INT9H	Far	CODE:042D
NOREMOVEMSG	Byte	CODE:0410
NOREMOVEMSG_LENGTH	Number	001D
NOTINSTALLEDMSG	Byte	CODE:03DA
OLD_INT1CHOFFSET	Word	CODE:03A4
OLD_INT1CHSEGMENT	Word	CODE:03A6
OLD_INT2FHOFFSET	Word	CODE:03A8
OLD_INT2FHSEGMENT	Word	CODE:03AA
OLD_INT9HOFFSET	Word	CODE:03A0
OLD_INT9HSEGMENT	Word	CODE:03A2
PRINTCHAR	Near	CODE:0726
PRINTDELAY	Number	0003
PRINTSIGNATURE	Near	CODE:061F
PRINTSIGNATURELINE	Near	CODE:060F
PRINTSTR	Near	CODE:072B
REMOVEDMSG	Byte	CODE:03F4
REMOVEDMSG_LENGTH	Number	001C
SAVEDSYMBOL	Byte	CODE:0390
SAVEFONT	Near	CODE:0546
SIGNATURELINE1	Byte	CODE:0117

SIGNATURELINE2	Byte	CODE:014B
SIGNATURELINE3	Byte	CODE:017F
SIGNATURELINELENGTH	Word	CODE:0115
SIGNATUREPRINTINGENABLED	Byte	CODE:0112
TABLEBOTTOM	Byte	CODE:01E7
TABLETOP	Byte	CODE:01B3
TOGGLECURSIVE	Near	CODE:04EA
TRANSLATEENABLED	Byte	CODE:0111
TRANSLATEFROM	Byte	CODE:010A
TRANSLATELENGTH	Byte	CODE:0110

TRANSLATETO	Byte	CODE:010D
TRUE	Number	00FF
_2FH_DEFAULT	Near	CODE:0565
_2FH_EXIT	Near	CODE:05DD
_ACTUALPRINT	Near	CODE:0636
_ALREADYINSTALLED	Near	CODE:06F7
_ALREADYINSTALLED2FH	Near	CODE:056A
_BLOCK	Near	CODE:04B6
_CHECKIGNORED	Near	CODE:04AA
_CHECKTRANSLATE	Near	CODE:04BE
_CHECKTRANSLATELOOP	Near	CODE:04CC
_DISPLAYHELP	Near	CODE:071A
_DONTPRINT	Near	CODE:0609
_EXIT	Near	CODE:0700
_EXITTOGGLECURSIVE	Near	CODE:0534
_F1	Near	CODE:0455
_F7	Near	CODE:0466
_F8	Near	CODE:0477
_F9	Near	CODE:048B
_GO	Near	CODE:0452
_INITKRBUFF	Near	CODE:066B
_LETSPRINT	Near	CODE:05FC
_NEXTCHAR	Near	CODE:070C
_NOTREMOVE	Near	CODE:05B8
_NOTTOPRINT	Near	CODE:060D
_PARAMPARSINGEND	Near	CODE:0725
_PRINTSTREND	Near	CODE:073B
_PRINTSTRLOOP	Near	CODE:072F
_QUIT	Near	CODE:04E1
_REMOVE	Near	CODE:06E3
_RESTORESSEMBOL	Near	CODE:0525
_SHIFTTABLE	Near	CODE:04FC
_SHOULDCONTINUE	Near	CODE:0688
_START	Near	CODE:0100
_TEST_FX	Near	CODE:0455
_TRANSLATE	Near	CODE:04D8
_TRANSLATEORIGNORE	Near	CODE:049C
_UNINSTALL	Near	CODE:056D
_UNLOADED	Near	CODE:05CC

Groups & Segments	Bit	Size	Align	Combine	Class
CODE	16	073E	Para	none	CODE